IN4120

Simple-9

How to pack many integers into one 32-bit word, and why it makes search faster

Based on Vo Ngoc Anh and Alistair Moffat: “Inverted Index Compression Using Word-Aligned Binary Codes”, Information Retrieval 8 (2005)

by Yasmin

Posting lists get huge

Inverted index (simplified)

search → 3, 7, 11, 12, 20, …

technology → 2, 7, 19, 40, …

the → 1, 2, 3, 4, 5, 6, …

Each term points to a sorted list of the IDs of the documents that contain it.

- **Common terms** appear in millions of documents, so their lists get very long.
- **32 bits per number** is wasteful when most of the numbers are small.
- **Compression** gives a smaller index: more of it fits in memory, and less has to be read from disk.

Store the gaps, not the document IDs

Document ID	3	7	11	12	20	22	23
Gap	3	4	4	1	8	2	1

The gap is the distance to the previous document: 7 − 3 = 4, 11 − 7 = 4, 12 − 11 = 1, and so on. The first number is stored as is.

The point: IDs can get large, but gaps are small, especially for common terms. Small numbers need few bits, and that is what Simple-9 exploits.

Pack many numbers into one word

- **The selector (4 bits):** Says how the data bits are split: how many numbers the word holds, and how many bits each number gets. 4 bits allow 16 values, but only 9 are used.
- **The data field (28 bits):** Holds the numbers themselves, all with the same number of bits. The smaller the numbers, the more of them fit in one word.

Table 1. Nine different ways of using 28 bits for flat binary codes (method Simple-9).

Selector	Number of codes	Length of each code (bits)	Number of unused bits
a	28	1	0
b	14	2	0
c	9	3	1
d	7	4	0
e	5	5	3
f	4	7	0
g	3	9	1
h	2	14	0
i	1	28	0

Encoding: How the encoder picks a mode

One word at a time

1. Look at the next numbers in the list.
2. Try the modes from most to fewest numbers: 28 × 1 bit, then 14 × 2 bits, then 9 × 3 bits, …
3. Pick the first mode where all the numbers fit.
4. Write the word, and continue with the rest of the list.

Weakness: one large integer raises the cost of its whole word

The gaps **1, 1, 1, 300, 1, 1**: 300 needs 9 bits, and a word cannot mix code lengths.

Word 1: [1, 1, 1] → 3 × 9 bits
Word 2: [300, 1, 1] → 3 × 9 bits

The ones get 9 bits each, but need only 1.

Why Simple-9 is fast to decode

BIT-ALIGNED

Elias codes

Decode bit by bit. The decoder must find where each variable-length code ends, which makes decoding slow.

BYTE-ALIGNED

Variable-byte

One byte at a time. A flag bit says whether the integer continues. Fast, but every integer takes at least 8 bits.

WORD-ALIGNED

Simple-9

A whole 32-bit word at a time. The selector gives the row, and all integers come out with simple bit operations.

Simple-9 handles several integers per fixed-size word: up to 28 from a single word. It decodes about as fast as variable-byte, but compresses better.

What Anh and Moffat found

1. **Much faster decoding.** Word-aligned codes decode a pointer roughly 5–15× faster than Golomb, and about as fast as byte-aligned codes (figure).

2. **Faster queries**. On the WSJ collection a query took 7.7 ms with Simple-9, against 12.7 ms with Golomb and 9.5 ms with byte-aligned codes.

3. **Compression in the middle**. On WSJ: Golomb 6.62, Simple-9 8.66 and byte-aligned 10.07 bits per pointer. Never worse than byte- or nibble-aligned codes.

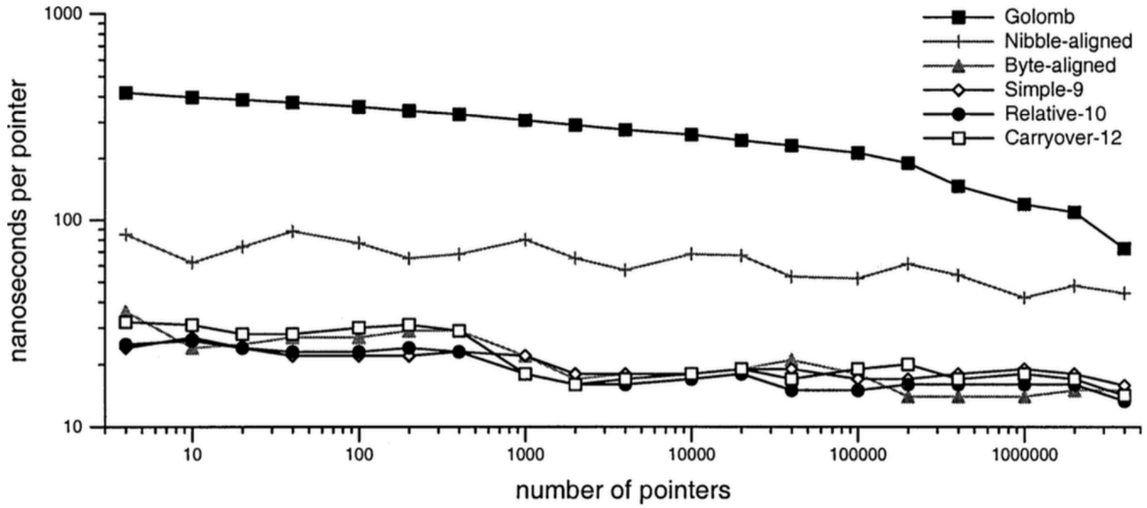


Figure 3. Decoding speed measured in CPU nanoseconds per pointer, as a function of the number of pointers randomly generated for hypothetical inverted lists for a collection containing $N = 10{,}000{,}000$ documents. These experiments were conducted on a 933 MHz Intel Pentium III with 1 GB RAM running Debian GNU/Linux.

Weak on short lists, and two fixes

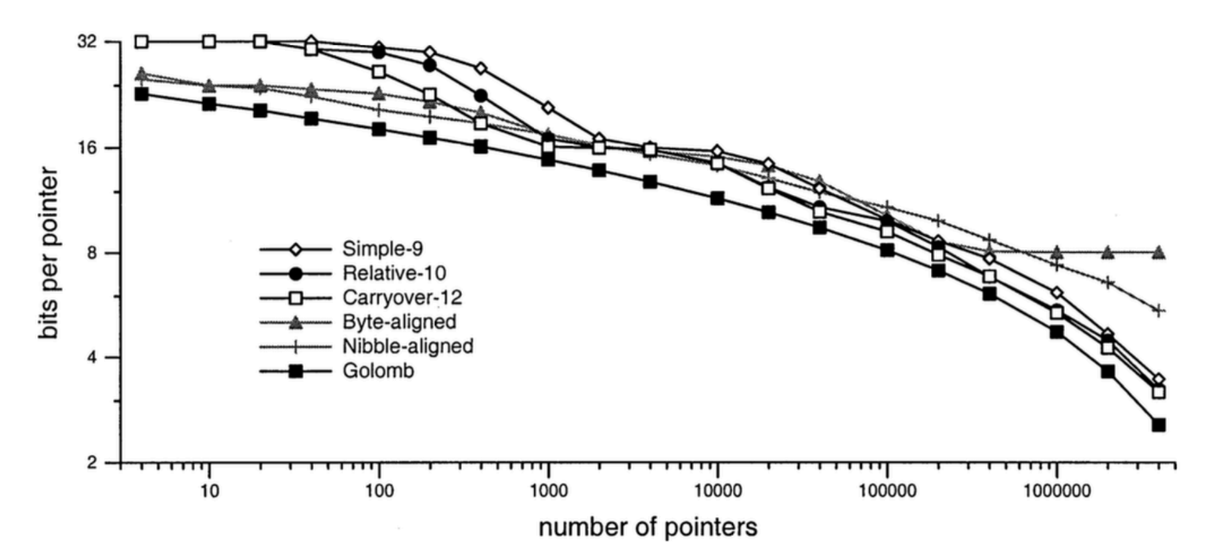


Figure 1. Bits per pointer for various integer coding methods, as a function of the number of pointers randomly generated for hypothetical inverted lists for a collection containing $N = 10{,}000{,}000$ documents. Compression is measured in average bits per pointer.

Improvement 1: Relative-10

Reduces the selector from 4 bits to 2 bits by encoding it relative to the previous word. This increases the available data space from 28 to 30 bits and provides ten possible encoding modes.

Improvement 2: Carryover-12

Uses otherwise unused bits in a word to store the selector for the next word. This allows the next word to use all 32 bits for data when a selector can be carried over.

Trading compression for speed

Benefits of a compressed index

- Reduced storage requirements.
- Less data transferred from disk.
- Faster query processing

Trade-offs

- Extra CPU work to decompress
- Worse compression than bit-aligned codes such as Golomb
- Fixed-width codes waste bits when integers vary a lot in size

Simple-9 in one sentence: better compression than variable-byte, and much faster decoding than bit-aligned codes such as Golomb.

References

- Anh, V. N., & Moffat, A. (2005). Inverted index compression using word-aligned binary codes. Information Retrieval, 8(1), 151–166.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.